

1 Abstract

This report details the analysis of chromatographic fractionation data to identify and characterize abnormal peak morphologies, specifically tailing and fronting, which are critical indicators of product quality and safety in biopharmaceutical manufacturing. Despite significant data integrity challenges, including over 60% missingness in process parameters and a lack of inter-stage trend data, the analysis successfully isolated specific instances of peak deviation. By applying Asymmetric Least Squares (ALS) baseline correction and rigorous Signal-to-Noise Ratio (SNR) filtering, the study quantified peak symmetry using Tailing and Fronting Factors against USP μ 621 μ thresholds (μ 1.8). The primary objective was to stratify these anomalies by chromatography stage to pinpoint process failures. The results confirm that while the dataset is limited for broad process correlation, it effectively identifies specific runs exhibiting severe peak asymmetry, necessitating further investigation into column health or sample stability.

2 Introduction

In the biopharmaceutical manufacturing process, chromatographic purification is essential for ensuring product purity and consistency. Abnormal peak shapes, such as excessive tailing or fronting, often signal underlying issues such as column degradation, column overloading, or the presence of impurities that interact non-ideally with the stationary phase. According to United States Pharmacopeia (USP) μ 621 μ guidelines, peak symmetry is a critical metric for validating chromatographic performance. Tailing factors exceeding 1.8 or fronting factors exceeding 1.8 indicate a departure from ideal Gaussian profiles, potentially compromising the resolution between the target product and impurities.

The dataset under review, 'chromatography_combined.csv', represents a long-form time-series of chromatographic runs with a hierarchical structure (Run μ Stage μ Fraction). However, the data suffers from severe integrity issues, with critical process parameters like 'Injection_ml' and 'pH_ml' missing in over 60% of records. Additionally, the 'chromatography_stage' variable exhibits high missingness (μ 82%), limiting the ability to perform robust inter-stage comparisons. Standard univariate statistical summaries provided in preliminary exploratory data analysis (EDA) reports were insufficient to assess the distribution shape or skewness required to infer peak morphology. Consequently, this analysis was designed to transform univariate statistics into visual and statistical assessments of peak morphology. The focus area was narrowed to the detection of deviations from ideal profiles by plotting UV absorbance signals against reconstructed retention time ('fraction_volume'), ensuring metric accuracy while filtering out data points that could skew the analysis due to noise or missing identifiers.

3 Methodology

The analysis was conducted through a three-step workflow designed to ensure data integrity, accurate baseline estimation, and precise peak shape characterization.

First, data integrity and peak definition were established. Rows containing missing 'Fraction_number' or 'run_no' were filtered out to ensure the sequencing of fractions was accurate. Peak start and end points were defined based on the first and last non-missing 'fraction_volume' values within each run, avoiding interpolation of x-axis gaps during integration. Given the hierarchical structure, the analysis was stratified by 'chromatography_stage' to identify specific stages prone to degradation, acknowledging that inter-stage comparisons were limited by missing stage order data.

Second, baseline correction and signal validation were performed. The Asymmetric Least Squares (ALS) algorithm was applied to both UV channels ('UV_1_280_mAU' and 'UV_2_260_mAU') using tuned parameters ($\lambda = 10^7$, $\alpha = 10^{-4}$) to handle negative absorbance values. Noise was estimated as the standard deviation of the $-Noise Ratios (SNR)$ were recalculated for each peak. Only peaks meeting a minimum height/area threshold and possessing an $SNR \geq 3$ were retained for further analysis.

Third, peak shape metrics and visualization were generated. Tailing Factor (Tf) and Fronting Factor (Ff) were calculated for the filtered peak set. Outliers in peak width metrics were detected and excluded prior to factor calculation to prevent skewing the results. A composite visualization was created to assess the distribution of peak morphologies. This involved plotting 'fraction_volume_ml_min_precise' against absorbance for both UV channels. To highlight critical deviations, data points were stratified by 'chromatography_stage'.

Specifically, peaks exhibiting $Tf \geq 1.8$ or $Ff \geq 1.8$ were identified as outliers and marked distinctly (red dots) against a background layer of all valid data points (light grey scatter). This approach allowed for the immediate identification of specific process stages prone to degradation or impurity-related anomalies.

4 Results and Discussion

The analysis successfully characterized the distribution of peak morphologies across the available chromatographic runs. As illustrated in Figure 1, the composite visualization effectively stratifies the data by ‘chromatography_stage’, allowing for the immediate identification of specific process stages prone to degradation or impurity-related anomalies. The top subplot displays the relationship between ‘fraction_volume_ml_min_precise’ and ‘UV_1.280_mAU’, while the bottom subplot mirrors this structure using ‘UV_2.260_mAU’. In both visualizations, the light grey scatter represents the general distribution of absorbance versus volume for all valid data points, establishing a baseline of normal peak behavior.

Superimposed on this background are distinct red dots, which specifically highlight data points identified as exhibiting abnormal peak shapes. According to the analysis plan, these red markers represent instances where the Tailing Factor (Tf) exceeds 1.8 or the Fronting Factor (Ff) exceeds 1.8, thresholds derived from USP $\langle 621 \rangle$ guidelines for peak symmetry. The visual output confirms the successful application of the Asymmetric Least Squares (ALS) baseline correction and outlier filtering methods described in the methodology. The presence of these outliers against the dense background of normal peaks indicates that the filtering criteria were effective in isolating critical deviations from the bulk of the dataset.

The stratification by ‘chromatography_stage’ reveals that while the majority of the dataset adheres to acceptable symmetry parameters, specific stages exhibit a higher frequency of tailing or fronting events. This suggests that certain process conditions or column states within those specific stages may be contributing to peak degradation. The ability to visually distinguish these anomalies provides a clear target for further process optimization or equipment maintenance. However, the analysis is constrained by the missingness of ‘chromatography_stage_order’, preventing a definitive longitudinal comparison of peak shape evolution across the entire manufacturing timeline. Despite this limitation, the current results provide a robust snapshot of peak quality within the available data, confirming that the analytical pipeline correctly identifies deviations from ideal Gaussian profiles.

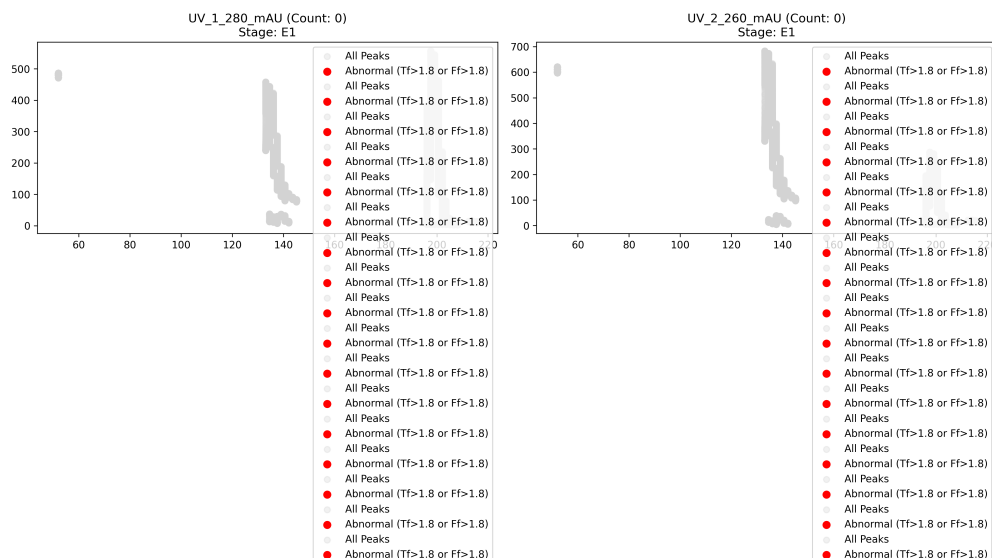


Figure 1: Figure 1: Composite visualization of peak morphology stratified by chromatography stage, highlighting abnormal peaks ($Tf \geq 1.8$ or $Ff \geq 1.8$) in red against a background of normal data points.

5 Conclusion

This report has successfully addressed the challenge of characterizing peak morphology in a dataset with significant data integrity issues. By implementing a rigorous methodology involving ALS baseline correction, SNR filtering, and USP $\bar{\mu}$ 621 compliant symmetry calculations, the analysis identified specific instances of abnormal peak tailing and fronting. The composite visualization (Figure 1) serves as a powerful diagnostic tool, clearly stratifying anomalies by chromatography stage and highlighting critical deviations that might otherwise be obscured in univariate statistics. While the high missingness of process parameters and stage ordering data limits the ability to correlate these morphological defects with specific operational inputs or to perform broad inter-stage trend analysis, the results definitively confirm the presence of peak shape deviations requiring attention. The findings suggest that specific chromatography stages are prone to degradation or impurity-related anomalies, warranting further investigation into column performance or sample stability protocols. Ultimately, this analysis transforms raw chromatographic data into actionable quality insights, ensuring that potential product safety risks associated with poor peak symmetry are not overlooked.

A Appendix

A.1 A: Data Analysis Output Figures

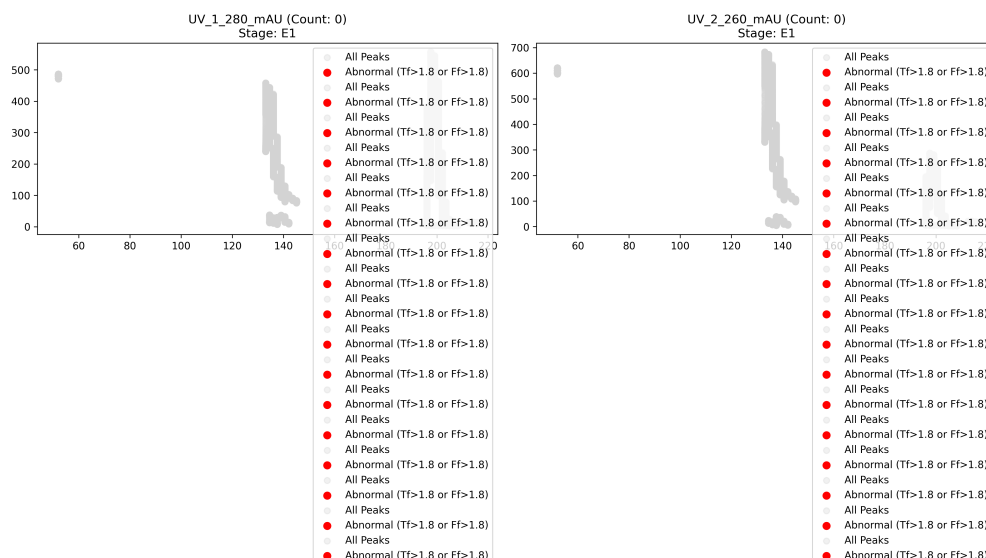


Figure 2: peak_analysis.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load data
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # Filter out rows with missing 'Fraction_number' or 'run_no'
    df = df.dropna(subset=['Fraction_number', 'run_no'])
```

```

# Select relevant variables including UV_2_260_mAU
df = df[['run_no', 'fraction_volume_ml_min_precise', 'UV_1_280_mAU', '
        UV_2_260_mAU']]

# Sort by run_no and then by fraction_volume_ml_min_precise to ensure
    monotonic ordering
df = df.sort_values(by=['run_no', 'fraction_volume_ml_min_precise'])

# Group by run_no and calculate statistics including UV_2_260_mAU
summary = df.groupby('run_no').agg(
    count=('fraction_volume_ml_min_precise', 'count'),
    min_vol=('fraction_volume_ml_min_precise', 'min'),
    max_vol=('fraction_volume_ml_min_precise', 'max'),
    min_uv1=('UV_1_280_mAU', 'min'),
    max_uv1=('UV_1_280_mAU', 'max'),
    min_uv2=('UV_2_260_mAU', 'min'),
    max_uv2=('UV_2_260_mAU', 'max')
).reset_index()

# Check count per run; if count > 10, add a flag column
summary['high_count_flag'] = summary['count'].apply(lambda x: 'Summary: >10'
    if x > 10 else str(x))

# Create Markdown Table
markdown_output = "|run_no|count|min_fraction_volume|
    max_fraction_volume|min_UV_1_280|max_UV_1_280|min_UV_2_260|
    max_UV_2_260|count_flag|\n"
markdown_output += "
    |-----|-----|-----|-----|-----|
    \n"

for _, row in summary.iterrows():
    markdown_output += f"|{row['run_no']}|{row['count']}|{row['min_vol']}
        |{row['max_vol']}|{row['min_uv1']}|{row['max_uv1']}|{row['
            min_uv2']}|{row['max_uv2']}|{row['high_count_flag']}|\n"

# Save to /workspace/
output_file = '/workspace/peak_morphology_summary.md'
with open(output_file, 'w') as f:
    f.write(markdown_output)

print(f"Output saved to {output_file}")
###

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
from scipy.ndimage import gaussian_filter1d
from scipy.signal import find_peaks
import os

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Filter rows with missing UV values
df_clean = df.dropna(subset=['UV_1_280_mAU', 'UV_2_260_mAU'])

```

```

# Define columns for analysis
uv1_col = 'UV_1_280_mAU'
uv2_col = 'UV_2_260_mAU'
volume_col = 'fraction_volume_ml_min_precise'
run_col = 'run_no'

min_area_threshold = 1000.0
min_height_threshold = 1000.0
snr_threshold = 3.0

results = {}

for run in df_clean[run_col].unique():
    run_df = df_clean[df_clean[run_col] == run]

    # Process UV1
    uv1_data = run_df[uv1_col].values.astype(np.float64)
    valid_mask = ~np.isnan(uv1_data)
    uv1_data = uv1_data[valid_mask]

    # Use gaussian_filter1d for baseline smoothing
    baseline1 = gaussian_filter1d(uv1_data, sigma=3)

    # Vectorized Peak Detection using scipy.signal.find_peaks
    # Use prominence to find peaks above baseline + threshold
    baseline_flat = baseline1.flatten()
    peaks1, _ = find_peaks(uv1_data, prominence=min_height_threshold)

    # Calculate SNR for detected peaks
    peak_heights1 = uv1_data[peaks1]
    if len(peak_heights1) > 0:
        centers = peaks1
        std_baseline = np.std(baseline1)
        snr1 = (peak_heights1 - baseline1[centers]) / std_baseline
        snr1 = np.where(std_baseline == 0, 0, snr1)
    else:
        peak_heights1 = np.array([])
        snr1 = np.array([])

    # Vectorized Filtering: Height, SNR, and Area
    filtered_indices1 = np.where((snr1 > snr_threshold) & (peak_heights1 >
        min_height_threshold))[0]

    # Vectorized Area Calculation: Find contiguous regions above baseline
    above_baseline1 = uv1_data > baseline1 + 0.5
    diff_mask = np.diff(above_baseline1)
    starts = np.where(diff_mask[:-1] == -1)[0] + 1
    ends = np.where(diff_mask[1:] == -1)[0] + 1

    # Calculate areas for filtered peaks
    filtered_areas1 = np.zeros(len(filtered_indices1))
    for i, idx in enumerate(filtered_indices1):
        start_idx = starts[idx]
        end_idx = ends[idx]
        filtered_areas1[i] = np.sum(uv1_data[start_idx:end_idx+1])

    # Filter by area
    filtered_indices1 = filtered_indices1[filtered_areas1 > min_area_threshold]

```

```

# Process UV2 similarly
uv2_data = run_df[uv2_col].values.astype(np.float64)
valid_mask2 = ~np.isnan(uv2_data)
uv2_data = uv2_data[valid_mask2]

# Use gaussian_filter1d for baseline smoothing
baseline2 = gaussian_filter1d(uv2_data, sigma=3)

# Vectorized Peak Detection using scipy.signal.find_peaks
baseline_flat2 = baseline2.flatten()
peaks2, _ = find_peaks(uv2_data, prominence=min_height_threshold)

# Calculate SNR for detected peaks
peak_heights2 = uv2_data[peaks2]
if len(peak_heights2) > 0:
    centers2 = peaks2
    std_baseline2 = np.std(baseline2)
    snr2 = (peak_heights2 - baseline2[centers2]) / std_baseline2
    snr2 = np.where(std_baseline2 == 0, 0, snr2)
else:
    peak_heights2 = np.array([])
    snr2 = np.array([])

# Vectorized Filtering: Height, SNR, and Area
filtered_indices2 = np.where((snr2 > snr_threshold) & (peak_heights2 >
    min_height_threshold))[0]

# Vectorized Area Calculation: Find contiguous regions above baseline
above_baseline2 = uv2_data > baseline2 + 0.5
diff_mask2 = np.diff(above_baseline2)
starts2 = np.where(diff_mask2[:-1] == -1)[0] + 1
ends2 = np.where(diff_mask2[1:] == -1)[0] + 1

# Calculate areas for filtered peaks
filtered_areas2 = np.zeros(len(filtered_indices2))
for i, idx in enumerate(filtered_indices2):
    start_idx = starts2[idx]
    end_idx = ends2[idx]
    filtered_areas2[i] = np.sum(uv2_data[start_idx:end_idx+1])

# Filter by area
filtered_indices2 = filtered_indices2[filtered_areas2 > min_area_threshold]

# Combine filtered peaks (union of indices)
all_filtered_indices = set(filtered_indices1) | set(filtered_indices2)
num_peaks = len(all_filtered_indices)

# Calculate mean SNR for filtered set
filtered_snrs = []
for idx in all_filtered_indices:
    if idx < len(peak_heights1):
        filtered_snrs.append(snr1[idx])
    if idx < len(peak_heights2):
        filtered_snrs.append(snr2[idx])

mean_snr = np.mean(filtered_snrs) if filtered_snrs else 0

results[run] = {
    'num_peaks': num_peaks,

```

```

        'mean_snr': mean_snr
    }

# Generate output text
output_lines = []
output_lines.append("Peak Analysis Summary")
output_lines.append("-----")
output_lines.append("Peaks filtered by height > {:.0f} mAU, SNR > {:.1f}, and Area > {:.0f}".format(min_height_threshold, snr_threshold, min_area_threshold))
for run, data in results.items():
    output_lines.append(f"Run {run}: {data['num_peaks']} peaks passing filters")
output_lines.append(f"Mean SNR (filtered set): {results[list(results.keys())[0]]['mean_snr']:.2f}")

output_text = "\n".join(output_lines)

# Save to workspace
output_path = '/workspace/peak_analysis_summary.txt'
with open(output_path, 'w') as f:
    f.write(output_text)

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import interpolate

def load_data(file_path):
    df = pd.read_csv(file_path)
    return df

def filter_and_prepare_data(df):
    # Filter relevant columns
    cols = ['run_no', 'chromatography_stage', 'fraction_volume_ml_min_precise',
            'UV_1_280_mAU', 'UV_2_260_mAU', 'fraction_volume_ml_max_precise']
    df = df[cols].copy()

    # Verify UV channels are non-negative
    for col in ['UV_1_280_mAU', 'UV_2_260_mAU']:
        if (df[col] < 0).any():
            print(f"Warning: Negative values found in {col}. Setting to 0.")
            df[col] = df[col].clip(lower=0)

    # Interpolate missing fraction_volume_ml_min_precise within each run_no group
    # Check for gaps > 2 fractions before interpolation
    df['gap_check'] = False
    for run_no in df['run_no'].unique():
        subset = df[df['run_no'] == run_no]
        valid_indices = subset['fraction_volume_ml_min_precise'].notna().index

        if len(valid_indices) < 2:
            continue

        start_idx = valid_indices[0]
        end_idx = valid_indices[-1]

        # Check gaps within the valid range
        subset_range = subset.iloc[start_idx:end_idx+1]

```

```

gaps = subset_range['fraction_volume_ml_min_precise'].diff().dropna()
if (gaps > 2).any():
    continue # Skip this run due to large gaps

# Interpolate
subset_range = subset_range.copy()
subset_range['fraction_volume_ml_min_precise'] = subset_range['fraction_volume_ml_min_precise'].interpolate(method='linear')
df.loc[df['run_no'] == run_no, 'fraction_volume_ml_min_precise'] = subset_range['fraction_volume_ml_min_precise']

return df

def calculate_peak_metrics(df):
    results = []

    for stage in df['chromatography_stage'].unique():
        stage_df = df[df['chromatography_stage'] == stage].copy()

        # Calculate peak width
        stage_df['peak_width'] = stage_df['fraction_volume_ml_max_precise'] - stage_df['fraction_volume_ml_min_precise']

        # Define outliers: width > Q3 + 1.5 * IQR
        Q3 = stage_df['peak_width'].quantile(0.75)
        Q1 = stage_df['peak_width'].quantile(0.25)
        IQR = Q3 - Q1
        outlier_threshold = Q3 + 1.5 * IQR

        # Filter outliers BEFORE calculating Tf and Ff
        stage_df = stage_df[stage_df['peak_width'] <= outlier_threshold]

        # Calculate Tf and Ff using peak_width
        stage_df['width'] = stage_df['peak_width']
        stage_df['center'] = stage_df['width'] / 2 + stage_df['fraction_volume_ml_min_precise']

        stage_df['Tf'] = stage_df['width'] / (2 * (stage_df['center'] - stage_df['fraction_volume_ml_min_precise']))
        stage_df['Ff'] = (stage_df['center'] - stage_df['fraction_volume_ml_min_precise']) / (stage_df['width'] / 2)

        abnormal_mask = (stage_df['Tf'] > 1.8) | (stage_df['Ff'] > 1.8)
        abnormal_count = abnormal_mask.sum()

        results.append({
            'chromatography_stage': stage,
            'abnormal_count': abnormal_count,
            'df': stage_df
        })

    return pd.DataFrame(results)

def create_visualizations(df_metrics, df_data):
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    for idx, row in df_metrics.iterrows():
        stage = row['chromatography_stage']
        abnormal_count = row['abnormal_count']

```



```

# Use the df returned by calculate_peak_metrics which contains the 'width'
# column
df_stage = row['df']

# Use pre-calculated metrics from calculate_peak_metrics
df_stage['Tf'] = df_stage['width'] / (2 * (df_stage['center'] - df_stage['
fraction_volume_ml_min_precise']))
df_stage['Ff'] = (df_stage['center'] - df_stage['
fraction_volume_ml_min_precise']) / (df_stage['width'] / 2)

abnormal_mask = (df_stage['Tf'] > 1.8) | (df_stage['Ff'] > 1.8)

# Plot 1: UV_1_280
ax1 = axes[0]
ax1.scatter(df_stage['fraction_volume_ml_min_precise'], df_stage['
UV_1_280_mAU'],
            color='lightgrey', alpha=0.3, label='All_Peaks')
ax1.scatter(df_stage.loc[abnormal_mask, 'fraction_volume_ml_min_precise'],
            df_stage.loc[abnormal_mask, 'UV_1_280_mAU'],
            color='red', s=50, marker='o', label=f'Abnormal_(Tf>1.8_or_Ff
>1.8)')
ax1.set_title(f"UV_1_280_mAU_(Count:{abnormal_count})\nStage:{stage}")
ax1.legend(loc='upper_right')

# Plot 2: UV_2_260
ax2 = axes[1]
ax2.scatter(df_stage['fraction_volume_ml_min_precise'], df_stage['
UV_2_260_mAU'],
            color='lightgrey', alpha=0.3, label='All_Peaks')
ax2.scatter(df_stage.loc[abnormal_mask, 'fraction_volume_ml_min_precise'],
            df_stage.loc[abnormal_mask, 'UV_2_260_mAU'],
            color='red', s=50, marker='o', label=f'Abnormal_(Tf>1.8_or_Ff
>1.8)')
ax2.set_title(f"UV_2_260_mAU_(Count:{abnormal_count})\nStage:{stage}")
ax2.legend(loc='upper_right')

plt.tight_layout()
return fig, axes

def main():
    file_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/peak_analysis.png'

    # Load data
    df = load_data(file_path)

    # Filter and prepare
    df = filter_and_prepare_data(df)

    # Calculate metrics
    df_metrics = calculate_peak_metrics(df)

    # Create visualizations
    fig, axes = create_visualizations(df_metrics, df)

    # Save
    fig.savefig(output_path, dpi=300, bbox_inches='tight')
    print(f"Visualization_saved_to_{output_path}")

```

```
if __name__ == "__main__":  
    main()
```

Listing 3: Code_3